Electiva 1 · Bases de Datos

Sharding Lab

Sharding horizontal y vertical con replicación y análisis CAP sobre PostgreSQL

Guía de ejecución, construcción, funcionamiento de la aplicación y conclusiones

Integrantes

Laura Urueña
Daniela Guevara
Luis Robelto
Sergio Gómez

Fecha: 12 de octubre de 2026

Contenido

1. Cómo ejecutar el proyecto desde el RAR

2. Cómo se construyó

3. Partes de la página

4. Base tecnológica y teórica

5. Detalles adicionales

6. Conclusiones

Anexo A. Endpoints de la API

Referencias

1. Cómo ejecutar el proyecto desde el RAR

El proyecto se entrega comprimido en un archivo .rar. Contiene todo lo necesario para levantar la aplicación completa: el código, los esquemas de base de datos y la configuración de Docker. No hay que instalar PostgreSQL ni Python: todo corre en contenedores.

1.1 Requisitos

Requisito	Detalle
Sistema operativo	Windows 10/11 (también funciona en macOS o Linux).
Docker Desktop	Instalado y abierto (en Windows con WSL 2 y la virtualización activada en la BIOS). Es lo único que hay que instalar.
Descompresor	WinRAR o 7-Zip para abrir el .rar.
Recursos	Unos 4 GB de RAM asignados a Docker (Docker Desktop → Settings → Resources) y varios GB de disco libres para las imágenes.
Puertos libres	8000 (web), 5441-5444 (primarios) y 5451-5454 (réplicas).
Internet	Solo la primera vez, para descargar las imágenes postgres:16 y python:3.12-slim.

1.2 Paso a paso

1. **Extraer el RAR.** Clic derecho → «Extraer aquí». Debe quedar una carpeta ShardingHorVer que contenga docker-compose.yml. Se recomienda una ruta corta y fuera de OneDrive, por ejemplo C:\proyectos\ShardingHorVer.
2. **Abrir Docker Desktop** y esperar a que indique que el motor está en ejecución (ícono verde).
3. **Abrir una terminal en esa carpeta.** En el Explorador de archivos, entrar a la carpeta, escribir cmd (o powershell) en la barra de direcciones y pulsar Enter.
4. **Levantar todo:**

```
docker compose up --build
```

5. **Esperar a que termine el arranque.** La primera vez tarda unos minutos. Está listo cuando en el registro aparece «Application startup complete» (API) y las réplicas muestran «started streaming WAL from primary». Los mensajes «Connection refused» al inicio son normales: la API arranca unos segundos antes que las bases de datos y reintenta.
6. **Abrir el navegador en** http://localhost:8000. La documentación interactiva de la API está en http://localhost:8000/docs.
7. **Pulsar «Sembrar 3000 clientes (reset)»** en la pestaña Arquitectura para cargar datos de prueba y ver el reparto.

1.3 Cómo detenerlo y reiniciarlo

Acción	Comando
Detener (conserva los datos)	Ctrl + C en la terminal, o en otra: docker compose down
Volver a arrancar (rápido)	docker compose up

Acción	Comando
Empezar de cero (borra los datos)	docker compose down -v y luego docker compose up --build
Ver el estado de los contenedores	docker compose ps (deben aparecer 9 en estado Up)
Ver el registro de un nodo	docker compose logs shard0-replica --tail 30

1.4 Qué contiene el RAR

```
ShardingHorVer/
├── docker-compose.yml        # 4 primarios + 4 réplicas + API (9 contenedores)
├── Dockerfile                # imagen de la API
├── requirements.txt          # dependencias de Python
├── app/                      # código de la aplicación
│   ├── main.py               #   API REST (FastAPI)
│   ├── db.py                 #   nodos, failover, replicación
│   ├── router.py             #   función de sharding
│   ├── seed.py               #   carga masiva de datos
│   └── static/index.html     #   interfaz web
├── sql/                      # esquemas de cada tipo de nodo
│   ├── shard_init.sql  perfil_init.sql  00_replication.sh
├── scripts/                  # replica-entrypoint.sh, demo.py, local_cluster.sh
├── tests/                    # 16 pruebas automatizadas
├── docs/                     # informe y capturas
└── README.md
```

Al generar el RAR no deben incluirse las carpetas .venv, .pgdata ni __pycache__ (son locales y pesadas).

1.5 Problemas frecuentes

Síntoma	Causa y solución
«Cannot connect to the Docker daemon»	Docker Desktop no está abierto. Abrirlo y esperar a que arranque.
«port is already allocated»	Otro programa usa el puerto (8000, 544x o 545x). Cerrarlo, o cambiar el número de la izquierda en el docker-compose.yml, por ejemplo "8001:8000".
Una réplica se reinicia con «bad interpreter» o «no such file»	Los scripts .sh quedaron con saltos de línea de Windows (CRLF). Abrir scripts/replica-entrypoint.sh y sql/00_replication.sh en VS Code, cambiar CRLF por LF (esquina inferior derecha), guardar y ejecutar docker compose down -v y luego up --build.
La réplica muestra «—» o menos filas que el primario	Aún está sincronizando (las réplicas aplican con 1 s de retraso a propósito). Esperar unos segundos.
Los contadores están en 0	No se han sembrado datos: pulsar «Sembrar 3000 clientes (reset)».
Todo se ve lento o Docker se cierra	Falta memoria: aumentar la asignada a Docker Desktop (Settings → Resources).

1.6 Ejecutar las pruebas automatizadas (opcional)

Con el sistema levantado y Python 3.11 o superior instalado, en otra terminal dentro de la carpeta del proyecto:

```
python -m venv .venv
.venv\Scripts\activate            # en Linux/macOS: source .venv/bin/activate
pip install -r requirements.txt
python -m pytest -q                 # 16 pruebas contra los nodos reales
python -m scripts.demo              # demostración por consola
```

Importante: las pruebas borran y vuelven a sembrar los datos, por lo que no deben ejecutarse en medio de una demostración.

2. Cómo se construyó

El proyecto se desarrolló por etapas, cada una con un resultado verificable. Esta sección resume el proceso y las decisiones que se tomaron en cada una.

Etapa	Qué se hizo	Resultado
1	Elegir la tecnología	PostgreSQL 16 (gratuito, replicación nativa) + Docker Compose + Python/FastAPI. Se descartó un motor con sharding automático para poder implementar y entender el reparto.
2	Diseñar el dominio y el reparto	Tienda en línea: clientes, pedidos y productos. Shard key = cliente_id; hash MD5 módulo 3; pedidos co-ubicados con su cliente; productos replicados; columnas pesadas del cliente al nodo vertical.
3	Escribir los esquemas SQL	sql/shard_init.sql (clientes_core, pedidos, pedido_items, productos) y sql/perfil_init.sql (clientes_perfil). Cada contenedor los ejecuta al crearse.
4	Implementar el router	app/router.py: shard_index(cliente_id, n) con MD5, determinista entre procesos.
5	Capa de datos y API	app/db.py (pools por nodo, alta con compensación, lecturas con failover, scatter-gather) y app/main.py (endpoints REST).
6	Carga masiva de datos	app/seed.py agrupa los inserts por shard y los envía en lote: 3000 clientes con ~6000 pedidos en pocos segundos.
7	Pruebas automatizadas	Las pruebas consultan cada nodo directamente para verificar la ubicación física de los datos; se confirmó el reparto, la separación vertical y los totales.
8	Interfaz web	Primera versión sencilla; después se rediseñó con cuatro pestañas, topología visual y rutas de consulta.
9	Replicación y CAP	Una réplica física por nodo (streaming replication), modo síncrono/asíncrono conmutable, failover de lectura y simulación de caídas. Se añadieron 8 pruebas más.
10	Contenedorización completa	docker-compose.yml con 9 contenedores y script de arranque de réplicas (pg_basebackup). Se probó en Windows con Docker Desktop.
11	Documentación	README, este informe y capturas de la aplicación en funcionamiento.

2.1 Dificultades encontradas y cómo se resolvieron

- **Datos que no aparecían al sembrar:** la interfaz mostraba las estadísticas antes de que terminara la carga. Se corrigió esperando la respuesta y mostrando los errores de la API.
- **Scripts con saltos de línea de Windows:** Git en Windows convierte los .sh a CRLF y los contenedores Linux fallan. Se añadió un archivo .gitattributes que fuerza LF.

- **Diferencia entre modos de replicación invisible en una red local:** el retraso real es de milisegundos, por lo que se configuró recovery_min_apply_delay = 1 s en las réplicas para poder observarla.
- **Simular fallos sin tocar los contenedores:** se implementó una marca de «nodo caído» por primario y réplica en la aplicación, y una prueba aparte con un primario realmente inalcanzable.

3. Partes de la página

La página se divide en un encabezado y cuatro pestañas. Cada una demuestra un concepto de la materia. Los números rojos de las figuras corresponden a las tablas que las siguen.

3.1 Pestaña ① Arquitectura

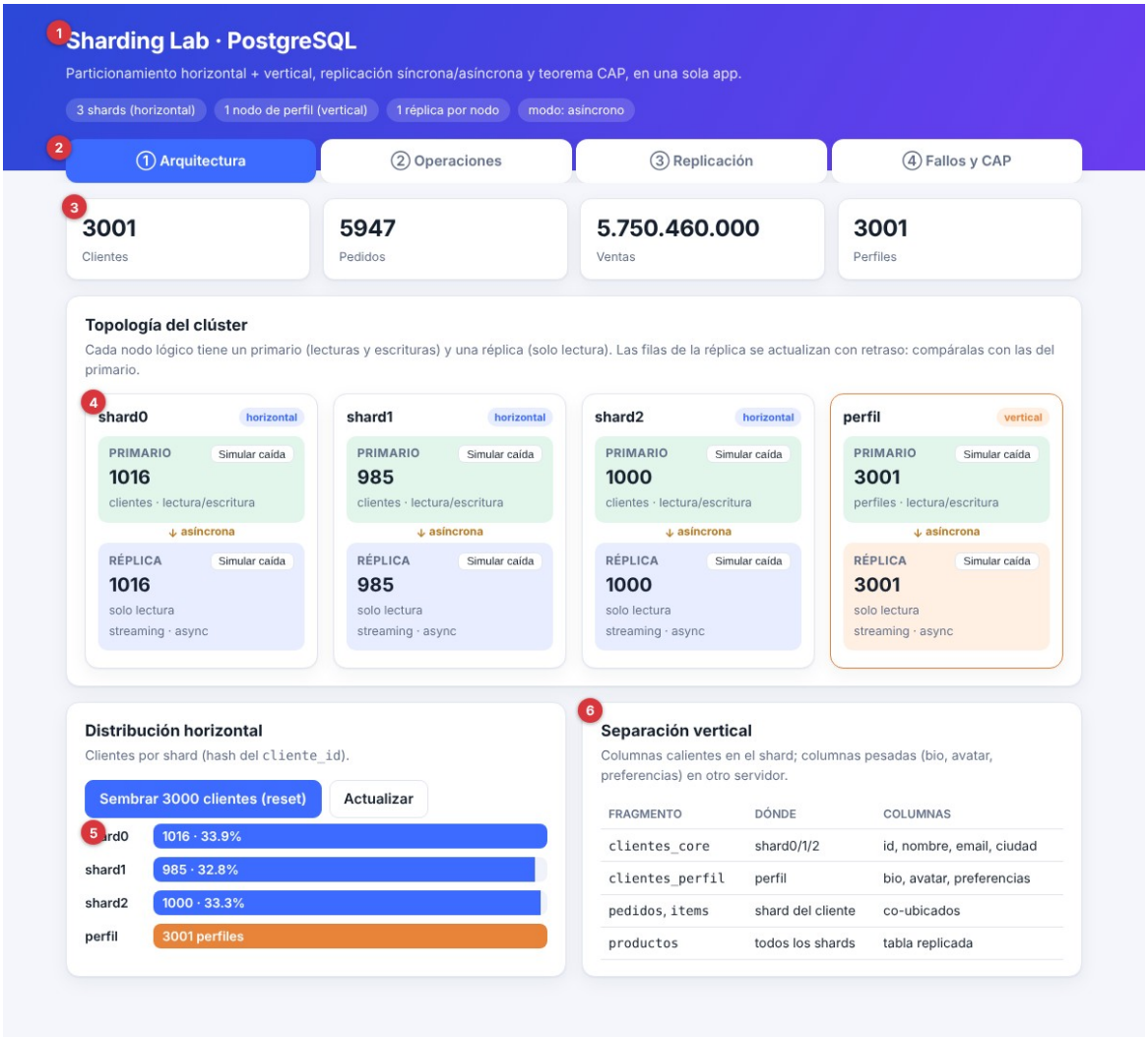


Figura 1. Pestaña Arquitectura con sus partes numeradas.

#	Parte	Qué muestra y qué concepto demuestra
1	Encabezado	Título y etiquetas con la configuración: 3 shards, 1 nodo de perfil, 1 réplica por nodo y el modo de replicación actual (síncrono o asíncrono).

#	Parte	Qué muestra y qué concepto demuestra
2	Pestañas	Navegación entre las cuatro secciones.
3	Indicadores	Totales de clientes, pedidos, ventas y perfiles. Se calculan sumando lo que responde cada shard (scatter-gather).
4	Topología del clúster	Un cuadro por nodo lógico (shard0, shard1, shard2 y perfil), con su primario (verde) y su réplica (azul). Cada cuadro muestra el número de filas; si la réplica va atrasada se indica cuántas faltan. El botón «Simular caída» apaga un nodo para la prueba de fallos.
5	Distribución horizontal	Barras con los clientes de cada shard y su porcentaje: evidencia el reparto uniforme del hash. Incluye el botón de sembrar datos.
6	Separación vertical	Resume qué columnas y tablas viven en cada nodo: core en los shards, perfil en el nodo vertical, productos replicados.

3.2 Pestaña ② Operaciones

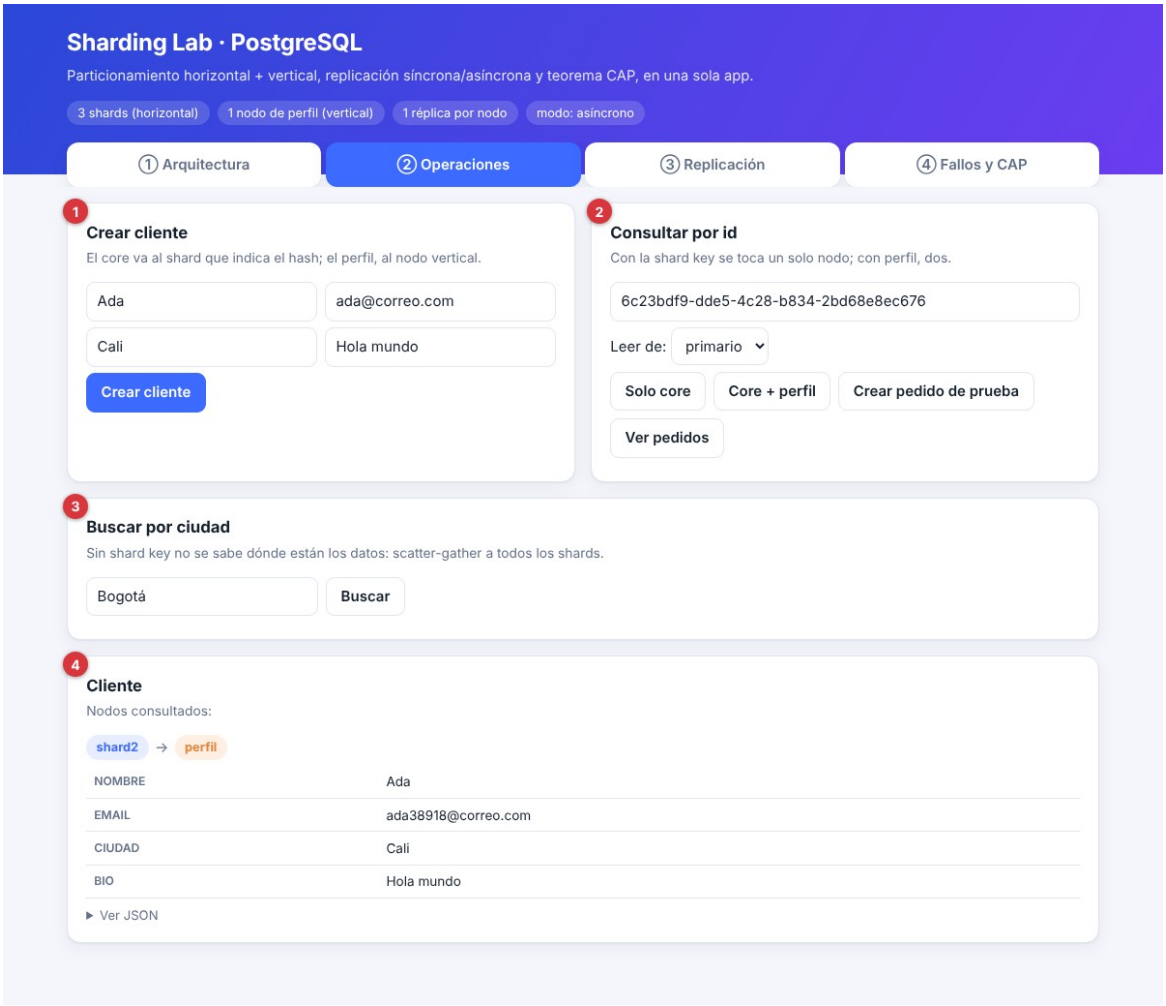


Figura 2. Pestaña Operaciones con una consulta de cliente con perfil.

#	Parte	Qué hace y qué demuestra
1	Crear cliente	Inserta el core en el shard que indica el hash y el perfil en el nodo vertical (partición mixta). Si falla el perfil, se deshace el core.
2	Consultar por id	Con la shard key se toca un solo nodo; «Core + perfil» añade el nodo vertical. «Leer de» permite elegir primario o réplica. También crea y lista pedidos, que quedan en el mismo shard del cliente.
3	Buscar por ciudad	Consulta sin shard key: la API pregunta a los tres shards y combina los resultados (scatter-gather).
4	Resultado	Muestra la ruta de nodos consultados (por ejemplo shard2 → perfil), los datos y, desplegable, el JSON completo. Si responde una réplica avisa que el dato puede estar desactualizado.

3.3 Pestaña ③ Replicación

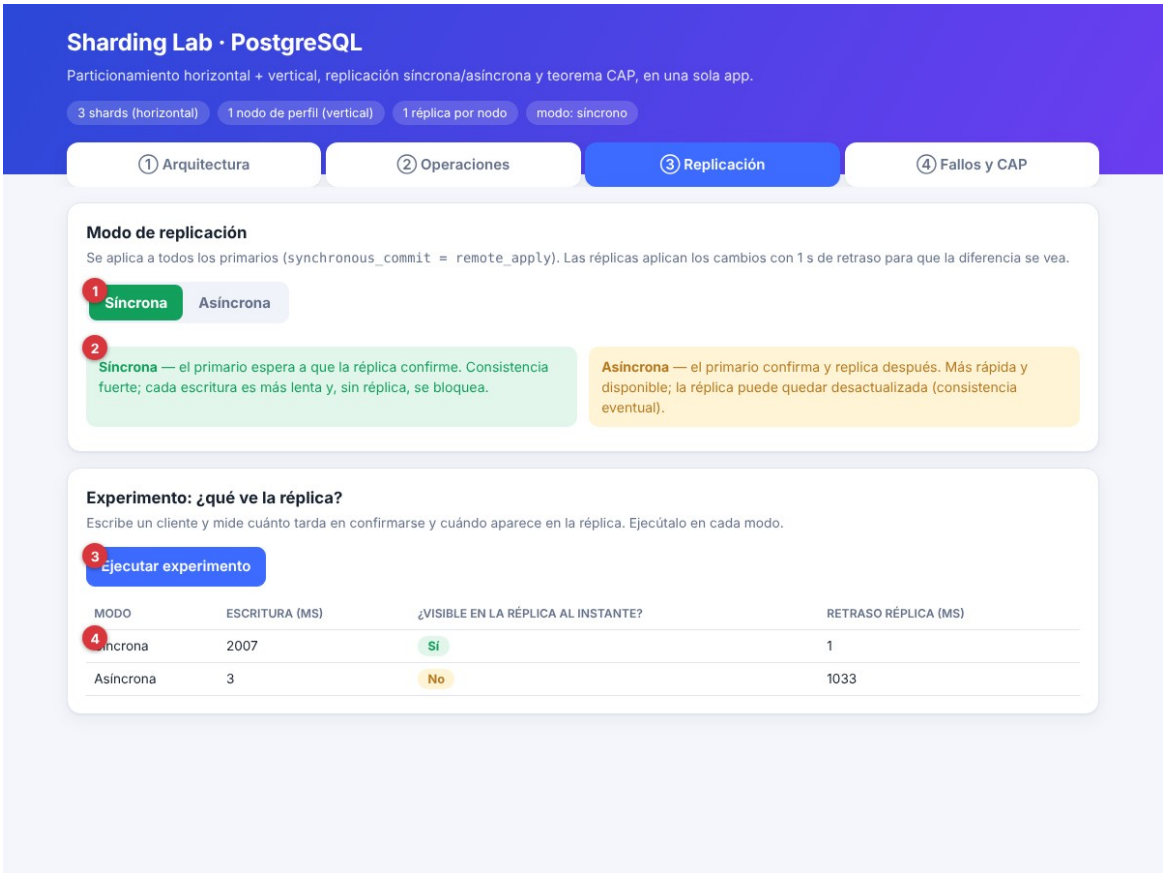


Figura 3. Pestaña Replicación con el resultado del experimento en ambos modos.

#	Parte	Qué hace y qué demuestra
1	Conmutador de modo	Cambia en caliente todos los primarios entre replicación síncrona (synchronous_commit = remote_apply) y asíncrona.
2	Explicación de cada modo	Síncrona: consistencia fuerte, escritura más lenta. Asíncrona: más rápida y disponible, consistencia eventual.

#	Parte	Qué hace y qué demuestra
3	Experimento	Escribe un cliente, mide cuánto tarda el commit y consulta enseguida la réplica.
4	Tabla de resultados	En asíncrono: escritura ~3 ms y la réplica lo ve ~1027 ms después. En síncrono: escritura ~2007 ms (dos commits, shard y perfil) y la réplica ya lo tiene.

3.4 Pestaña ④ Fallos y CAP

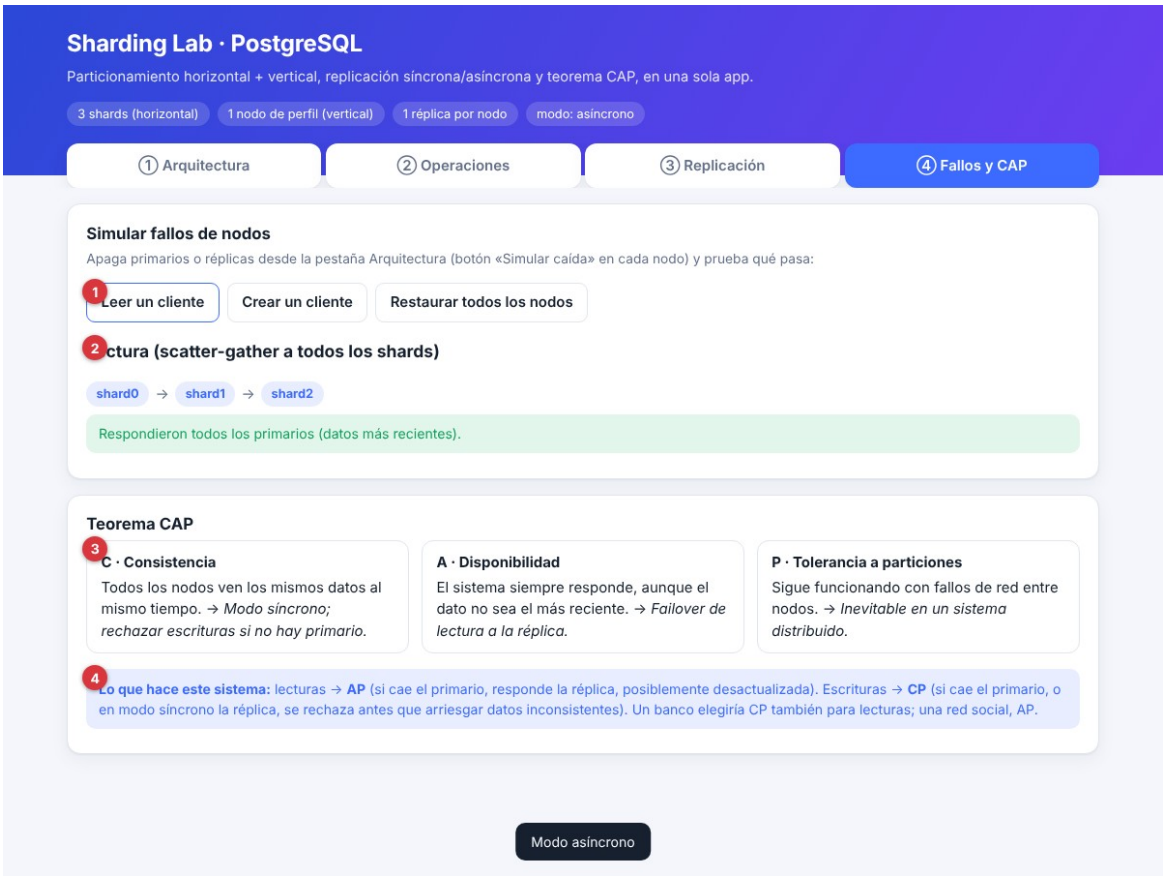


Figura 4. Pestaña Fallos y CAP.

#	Parte	Qué hace y qué demuestra
1	Botones de prueba	«Leer un cliente», «Crear un cliente» y «Restaurar todos los nodos». Se usan después de apagar nodos en la pestaña Arquitectura.
2	Resultado	Lectura: qué nodos respondieron; si fue una réplica, aparece la etiqueta AP. Escritura: si se rechazó, aparece CP con la causa.
3	C, A y P	Definición de cada propiedad y cómo la trata el sistema.
4	Conclusión del sistema	Lecturas AP (responde la réplica, quizá con dato antiguo) y escrituras CP (se rechazan antes que arriesgar inconsistencia).

Ejemplo de la prueba de fallos: con el primario de shard0 y el nodo de perfil apagados, la lectura sigue respondiendo desde shard0-replica con la etiqueta AP, y la creación de un cliente se rechaza con la etiqueta CP.

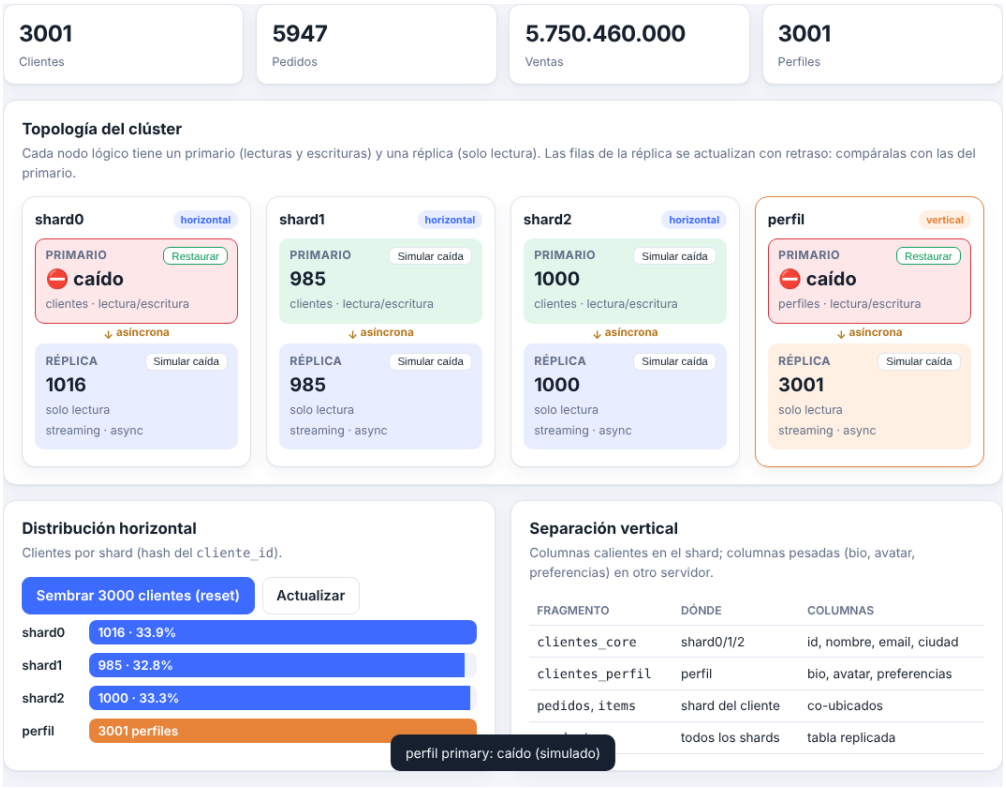


Figura 5. Primarios de shard0 y perfil caídos (simulación): sus réplicas siguen activas.

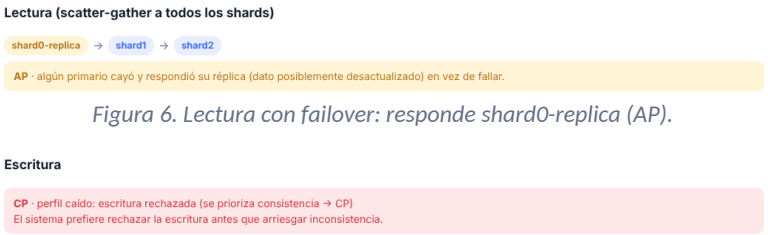


Figura 6. Lectura con failover: responde shard0-replica (AP).

Figura 7. Escritura rechazada para proteger la consistencia (CP).

4. Base tecnológica y teórica

4.1 Tecnologías utilizadas

Capa	Tecnología	Por qué
Base de datos	PostgreSQL 16	Gratuita y de código abierto, muy usada en la industria, con replicación por streaming nativa y modo síncrono configurable. 8 instancias independientes.
Contenedores	Docker y Docker Compose	Levantan los 9 servicios con un solo comando y hacen el entorno reproducible en cualquier equipo.

Capa	Tecnología	Por qué
Servidor / API	Python 3.12, FastAPI, Uvicorn	API REST ligera con documentación automática (/docs). Aquí vive el router de sharding.
Acceso a datos	psycopg 3 y psycopg_pool	Controlador de PostgreSQL con un pool de conexiones por nodo.
Interfaz	HTML, CSS y JavaScript	Sin librerías externas: funciona sin internet y se sirve desde la misma API.
Pruebas	pytest	Verifica contra los nodos reales, no contra simulaciones.

4.2 Conceptos teóricos aplicados

Concepto	Definición	Dónde se ve en el proyecto
Partición horizontal (sharding)	Divide las filas de una tabla en fragmentos repartidos entre nodos.	clientes_core, pedidos y pedido_items repartidos en 3 shards por hash del cliente.
Partición vertical	Divide las columnas de una tabla en nodos distintos.	Bio, avatar y preferencias en el nodo perfil; nombre, email y ciudad en los shards.
Partición mixta	Combina horizontal y vertical.	El cliente completo = core (shard según hash) + perfil (nodo vertical).
Shard key	Columna que decide en qué fragmento va cada fila.	cliente_id: toda consulta «de un cliente» toca un solo nodo.
Función hash	Convierte la clave en un número uniforme para repartir sin hot-spots.	shard = MD5(cliente_id) mod 3.
Co-ubicación	Guardar juntos los datos que se consultan juntos.	Los pedidos viven en el shard de su cliente: transacciones locales.
Tabla replicada	Copia completa de una tabla pequeña en cada nodo.	productos existe en los tres shards: sin joins entre shards.
Scatter-gather	Consultar a todos los nodos y combinar resultados.	Búsqueda por ciudad y estadísticas.
Replicación síncrona	El primario espera la confirmación de la réplica antes de responder.	Modo síncrono: consistencia fuerte, escritura más lenta.
Replicación asíncrona	El primario responde y replica después.	Modo asíncrono: rápido, consistencia eventual.
Teorema CAP	No se pueden garantizar a la vez consistencia, disponibilidad y tolerancia a particiones.	Lecturas AP con failover; escrituras CP.
Saga / compensación	Sin transacción distribuida, si un paso falla se deshacen los anteriores.	Alta de cliente entre shard y nodo perfil.

4.3 Sistemas CP y AP, y relación con CockroachDB y Vitess

Un sistema CP prioriza datos correctos y puede rechazar solicitudes ante fallos (ejemplo: un banco); uno AP siempre responde aunque el dato pueda estar desactualizado (ejemplo: una red social). La combinación CA solo funciona si la red nunca falla, por lo que no es realista en un sistema distribuido.

CockroachDB (SQL distribuido) y Vitess (MySQL distribuido) implementan estas ideas de forma nativa: reparten, replican y rebalancean los datos automáticamente. En este proyecto, con PostgreSQL estándar, esa lógica se implementó a mano en la aplicación. Eso permite ver y validar cada mecanismo, y deja claro qué automatizan esos motores.

5. Detalles adicionales

5.1 Validación: 16 pruebas automatizadas

Las pruebas se ejecutan contra el clúster real y todas pasan. Las de ubicación no usan la API: consultan cada nodo directamente.

Grupo	Qué se comprueba
Router (1)	Determinismo y reparto uniforme (±4 % con 30 000 ids).
Horizontal (2)	Cada fila está solo en el shard que le corresponde; los pedidos están con su cliente.
Vertical (2)	Las columnas pesadas no están en los shards; el nodo perfil no tiene clientes_core; la compensación funciona si cae el perfil.
Enrutamiento y agregación (3)	1 nodo por lectura caliente, 2 con perfil; totales y búsqueda por ciudad coinciden con la suma manual; un pedido inválido no deja basura.
Replicación (3)	Réplicas en solo lectura; asíncrona converge con retraso; síncrona ya tiene el dato al responder.
Fallos y CAP (5)	Failover de lectura, rechazo de escritura, bloqueo en modo síncrono sin réplica y failover ante un primario realmente inalcanzable.

5.2 Mediciones

Medición (6000 clientes, 11 880 pedidos)	Resultado
Reparto horizontal	1984 · 2023 · 1993 clientes por shard (menos de 2,5 % de desviación)
Tabla caliente frente a fría	clientes_core 528 kB · clientes_perfil 2416 kB (≈ 4,6 veces más pesada)
Lectura por id (1 nodo)	0,47 ms
Lectura por id con perfil (2 nodos)	0,96 ms
Búsqueda por ciudad (3 nodos)	3,20 ms
Escritura asíncrona / síncrona	~3 ms / ~2007 ms

La consulta con la shard key es unas siete veces más barata que el scatter-gather, lo que justifica la elección de cliente_id como clave.

5.3 Limitaciones

- El reparto está en la aplicación; CockroachDB o Vitess lo hacen de forma nativa.
- Cambiar el número de shards con «hash módulo N» obliga a mover casi todos los datos; en producción se usaría consistent hashing.
- El email es único solo dentro de cada shard; haría falta una tabla índice global.
- No hay commit distribuido: entre shard y perfil se usa compensación, con una ventana breve de inconsistencia.
- Si cae un primario no se promueve su réplica automáticamente (herramientas como Patroni lo harían).
- Las caídas son simuladas y el retraso de 1 s de las réplicas es artificial, ambos con fines demostrativos.
- Las credenciales (app/app) y la ausencia de autenticación son solo para laboratorio; no deben usarse así en producción.

5.4 Posibles mejoras

- Consistent hashing y un script de rebalanceo al agregar shards.
- Promoción automática de réplicas (failover de escritura).
- Tabla de búsqueda global para consultas por email sin scatter-gather.
- Migrar a CockroachDB o Vitess para comparar con el reparto manual.

6. Conclusiones

1. Se implementó y validó un sistema distribuido que combina partición horizontal (hash por cliente), vertical (columnas pesadas en un nodo aparte) y mixta, con los datos verificados físicamente en cada nodo.
2. La elección de la shard key define el costo de cada consulta: con la clave se toca un nodo (0,47 ms); sin ella, todos (3,20 ms). Por eso conviene elegirla según las consultas más frecuentes.
3. Separar las columnas pesadas mantiene liviana la tabla que se consulta más y permite leer el perfil solo cuando hace falta, a cambio de un salto de red adicional.
4. La replicación síncrona compra consistencia fuerte al precio de latencia (~2 s frente a ~3 ms en el experimento) y de menor disponibilidad; la asíncrona hace lo contrario. Medirlo sobre el mismo sistema deja ver el trade-off de la teoría.
5. El teorema CAP dejó de ser abstracto: se tradujo en decisiones comprobables, con lecturas AP (responde la réplica) y escrituras CP (se rechazan antes que arriesgar inconsistencia).
6. Construir el router a mano mostró qué automatizan motores como CockroachDB y Vitess, y qué costos (rebalanceo, transacciones distribuidas, unicidad global) siguen presentes en cualquier solución.
7. Usar Docker permitió que un sistema de nueve servicios se ejecute con un solo comando y se reproduzca en cualquier equipo, algo clave para una entrega comprimida en un RAR.

Anexo A. Endpoints de la API

Método	Ruta	Descripción
POST	/clientes	Crea un cliente: core al shard por hash, perfil al nodo vertical.
GET	/clientes/{id}?perfil=&lectura=	Consulta por id; opcionalmente con perfil y desde primario o réplica.
GET	/clientes?ciudad= o ?email=	Búsqueda sin shard key (scatter-gather).
DELETE	/clientes/{id}	Elimina el cliente en ambos fragmentos.
POST	/pedidos	Crea un pedido en el shard del cliente (transacción local).
GET	/clientes/{id}/pedidos	Pedidos del cliente (un solo shard).
GET	/stats	Totales y estadísticas por shard.
GET	/topologia	Estado de cada primario y réplica, filas y modo de replicación.
POST	/replicacion/modo	Cambia entre síncrona y asíncrona.
POST	/replicacion/demo	Experimento de latencia y retraso de la réplica.
POST	/nodos/{nombre}/{rol}?caido=	Simula la caída o restauración de un nodo.
POST	/demo/seed?n=&reset=	Carga masiva de datos de prueba.

Referencias

- PostgreSQL 16, High Availability, Load Balancing, and Replication. postgresql.org/docs/16/high-availability.html
- PostgreSQL 16, Write Ahead Log: synchronous_commit y recovery_min_apply_delay. postgresql.org/docs/16/runtime-config-wal.html
- Docker Compose, documentación oficial. docs.docker.com/compose
- Brewer, E. (2000). Towards Robust Distributed Systems. PODC. Gilbert, S. y Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services.
- Apuntes de clase: Bases de Datos Distribuidas (replicación, particionamiento, teorema CAP, CockroachDB y Vitess).